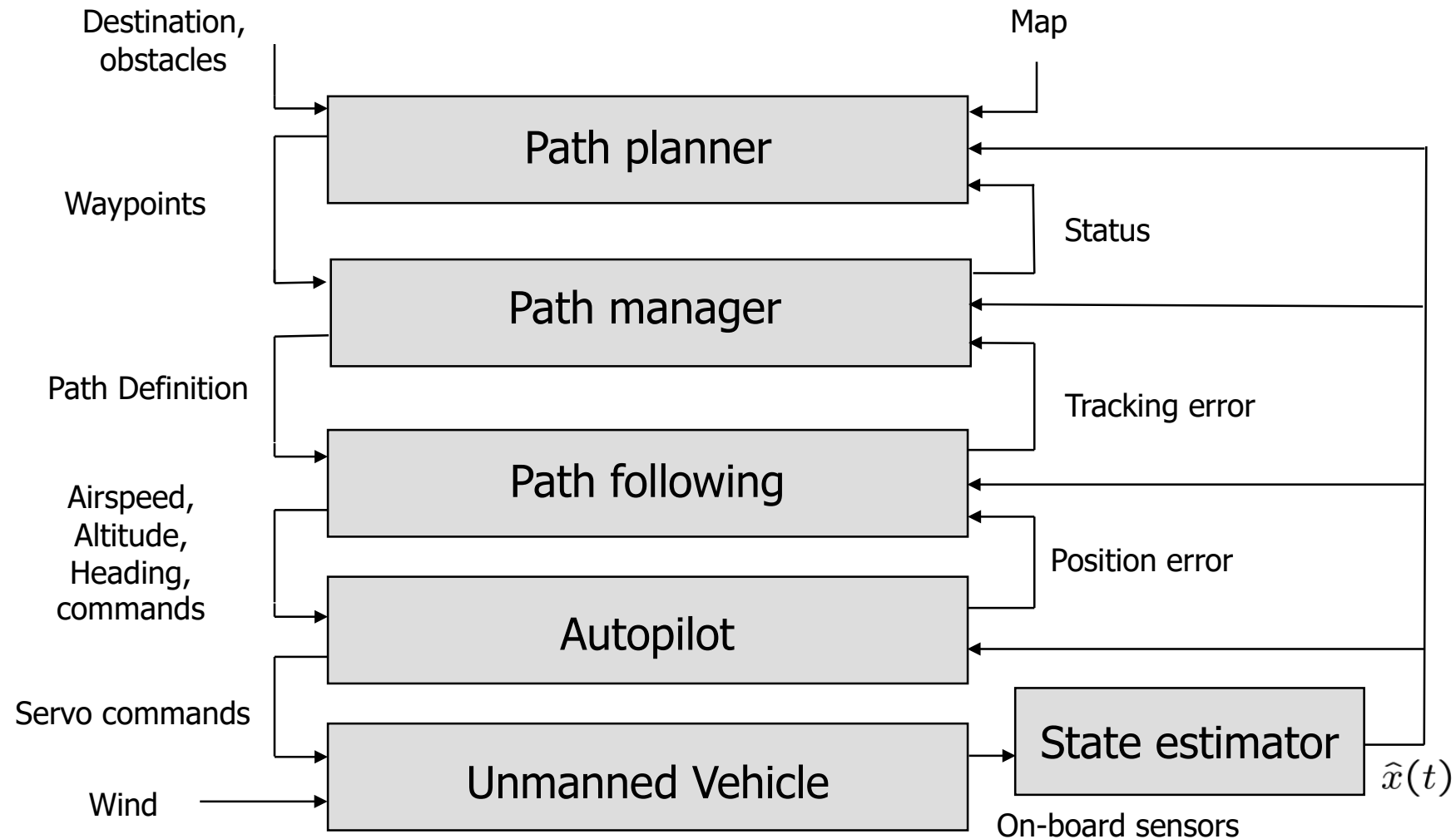


A yellow glider is shown in flight, positioned on the left side of the frame. The background features a vast, flat landscape with a field of mature corn in the foreground. In the distance, there is a small cluster of farm buildings, including a white barn and a windmill, silhouetted against the horizon. The sky is filled with large, billowing white and grey clouds, with some sunlight breaking through, creating a dramatic and atmospheric scene.

Лекция 6

Синтез автопилота

Архитектура



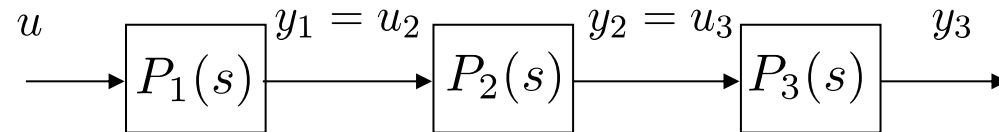
Варианты

Различные варианты проектирования (синтеза) автопилота

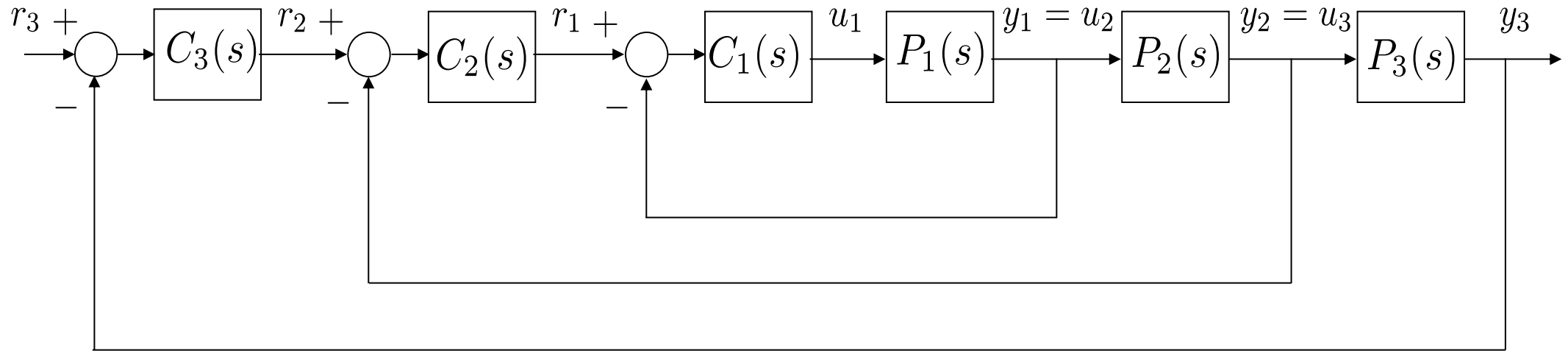
- Последовательное замыкание контуров
- Управление полной энергией
- LQR-управление

Последовательное замыкание контуров

Разомкнутая система

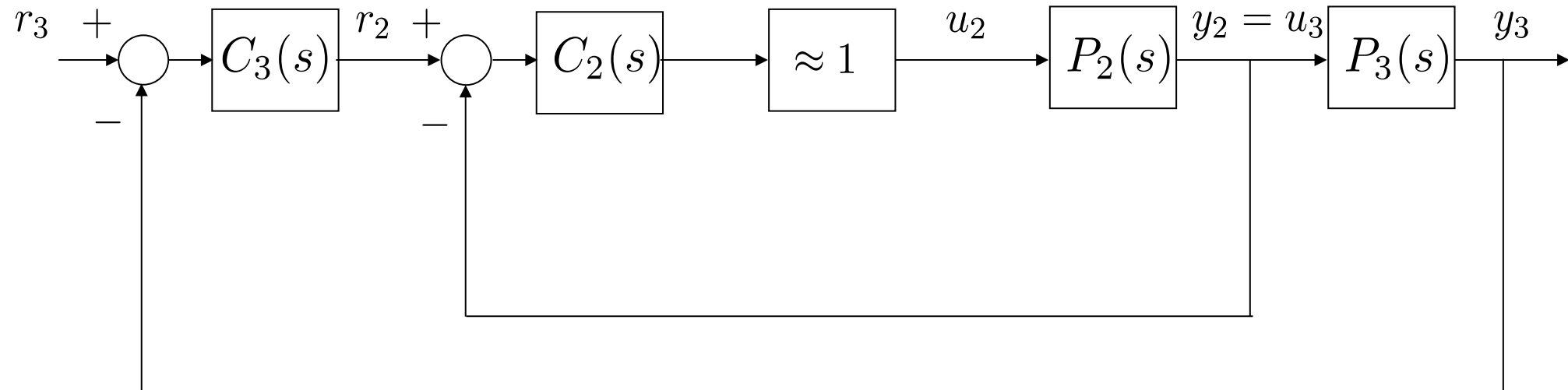


Замкнутая система



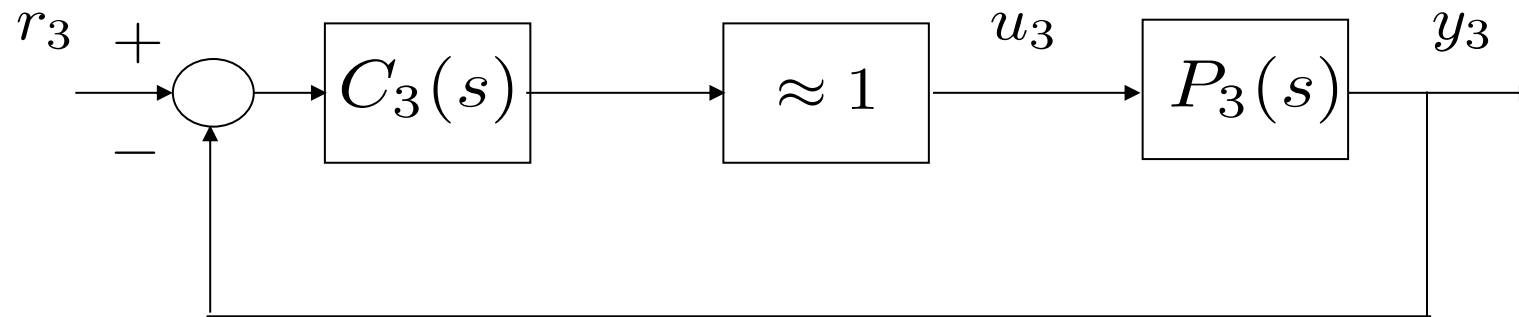
SLC: Замыкание внутреннего контура

На частотах ниже полосы пропускания внутреннего контура аппроксимируйте передаточную функцию замкнутой системы как 1, затем проектируйте средний контур



SLC: Замыкание двух контуров

На частотах ниже полосы пропускания среднего контура аппроксимируйте передаточную функцию замкнутой системы как 1, затем проектируйте внешний контур

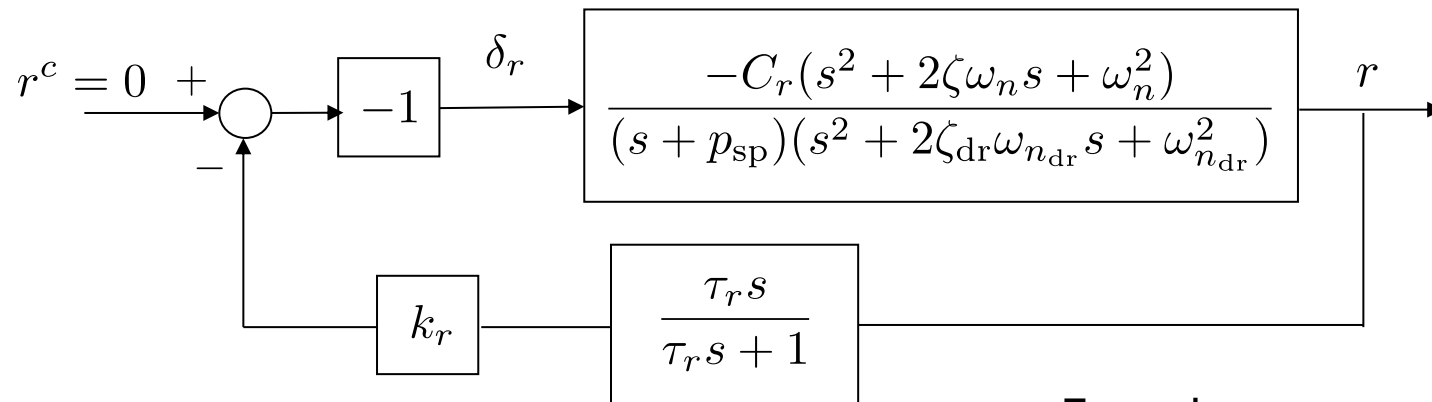
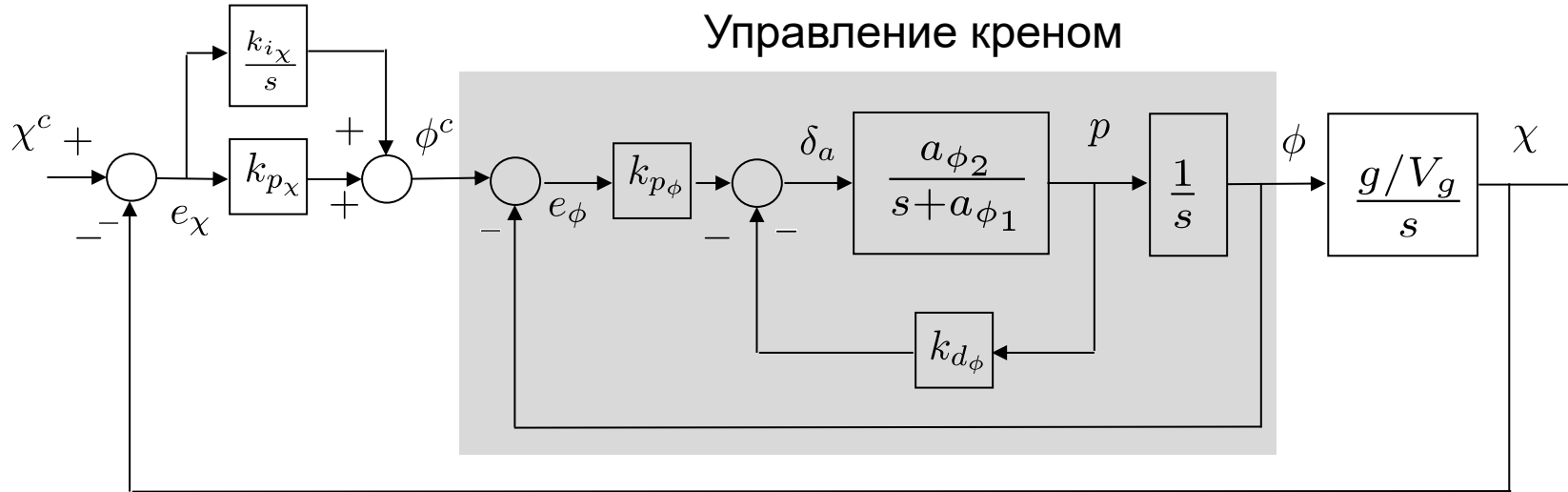


Ключевая идея: каждый последующий контур должен иметь меньшую полосу пропускания — обычно в 10 и более раз.

Автопилот бокового движения

Управление курсом

Управление креном



Демпфер рыскания

Автопилот крена

$$H_{\phi/\phi^c}(s) = \underbrace{\frac{k_{p\phi} a_{\phi_2}}{s^2 + (a_{\phi_1} + a_{\phi_2} k_{d\phi})s + k_{p\phi} a_{\phi_2}}}_{\text{ПФ замкнутой системы}} = \underbrace{\frac{\omega_{n\phi}^2}{s^2 + 2\zeta_{\phi}\omega_{n\phi}s + \omega_{n\phi}^2}}_{\text{Каноническая ПФ 2-го порядка}}$$

Параметры проектирования: $\omega_{n\phi}$ и ζ_{ϕ}

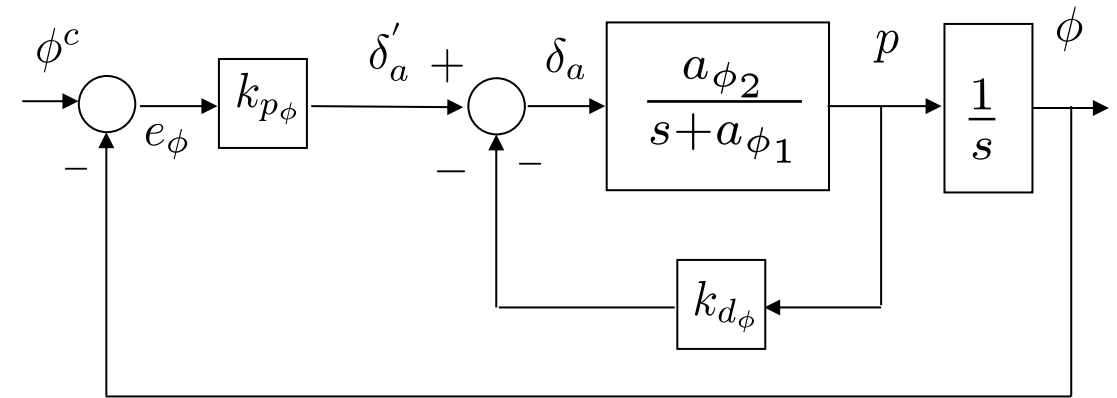
Коэффициенты усиления:

$$k_{p\phi} = \frac{\omega_{n\phi}^2}{a_{\phi_2}}$$

$$k_{d\phi} = \frac{2\zeta_{\phi}\omega_{n\phi} - a_{\phi_1}}{a_{\phi_2}}$$

Реализация:

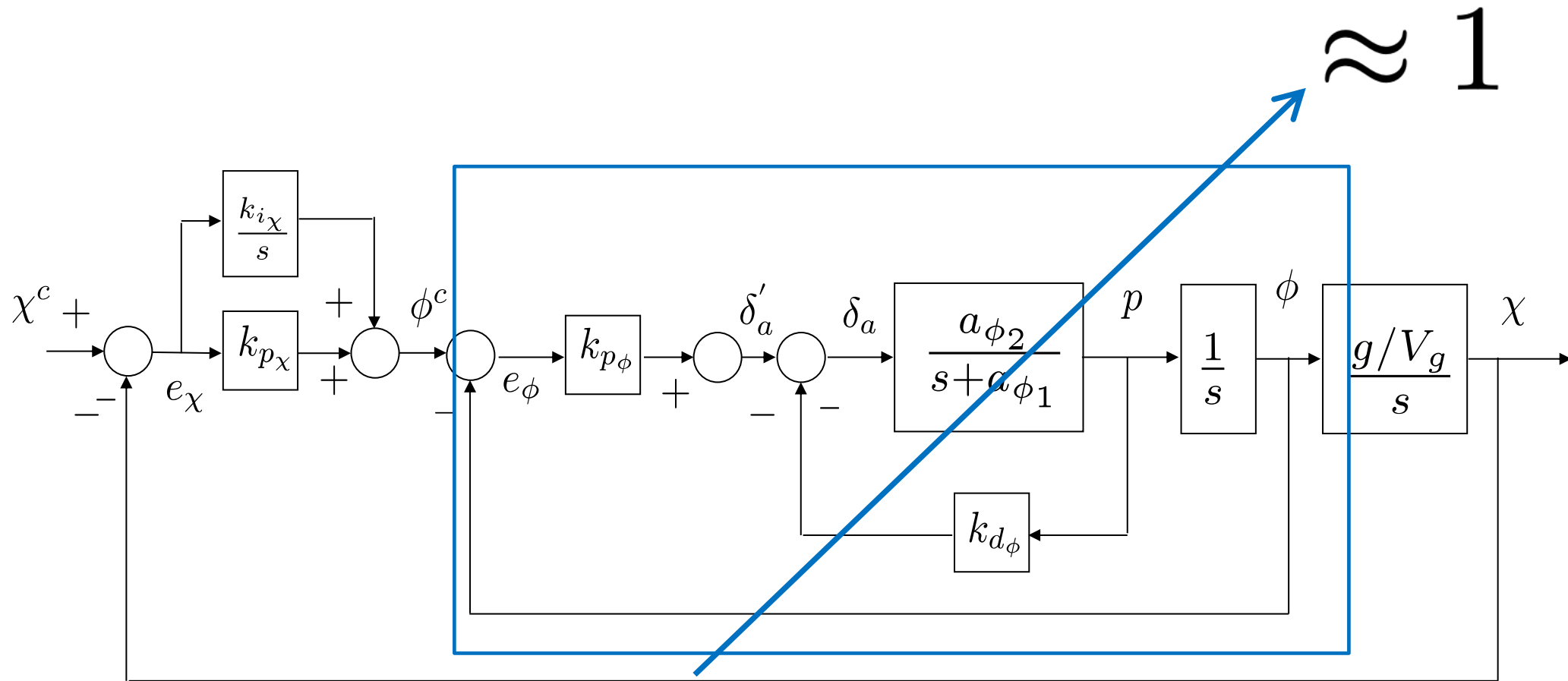
$$\delta_a(t) = k_{p\phi}(\phi^c(t) - \phi(t)) - k_{d\phi}p(t).$$



Автопилот крена

- В книге предлагается использовать интегратор по крену в контуре крена для компенсации установившейся ошибки, вызванной возмущениями.
- Наше текущее предложение — не использовать интегратор во внутренних контурах, включая контур крена.
 - Интеграторы вносят запаздывание и неустойчивость → не лучшая идея для самых внутренних контуров.
 - Интегратор будет использоваться в контуре курса для компенсации установившихся ошибок.

Автопилот бокового движения



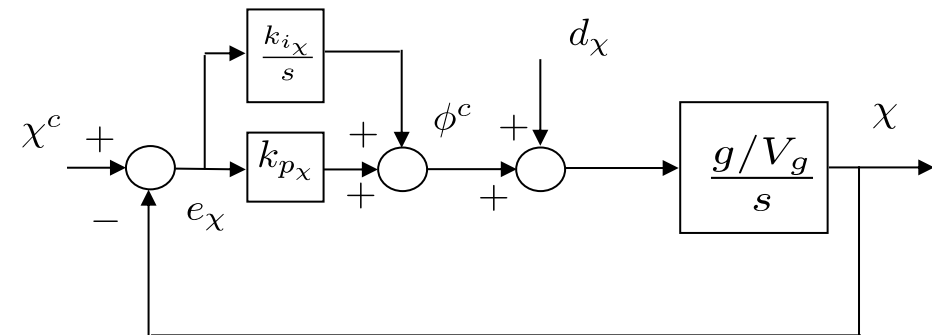
Контур удержания курса

Для контура курса обратите внимание на наличие входного возмущения. Используя ПИ-регулятор для курса, реакция на команду курса и возмущение задаётся выражением

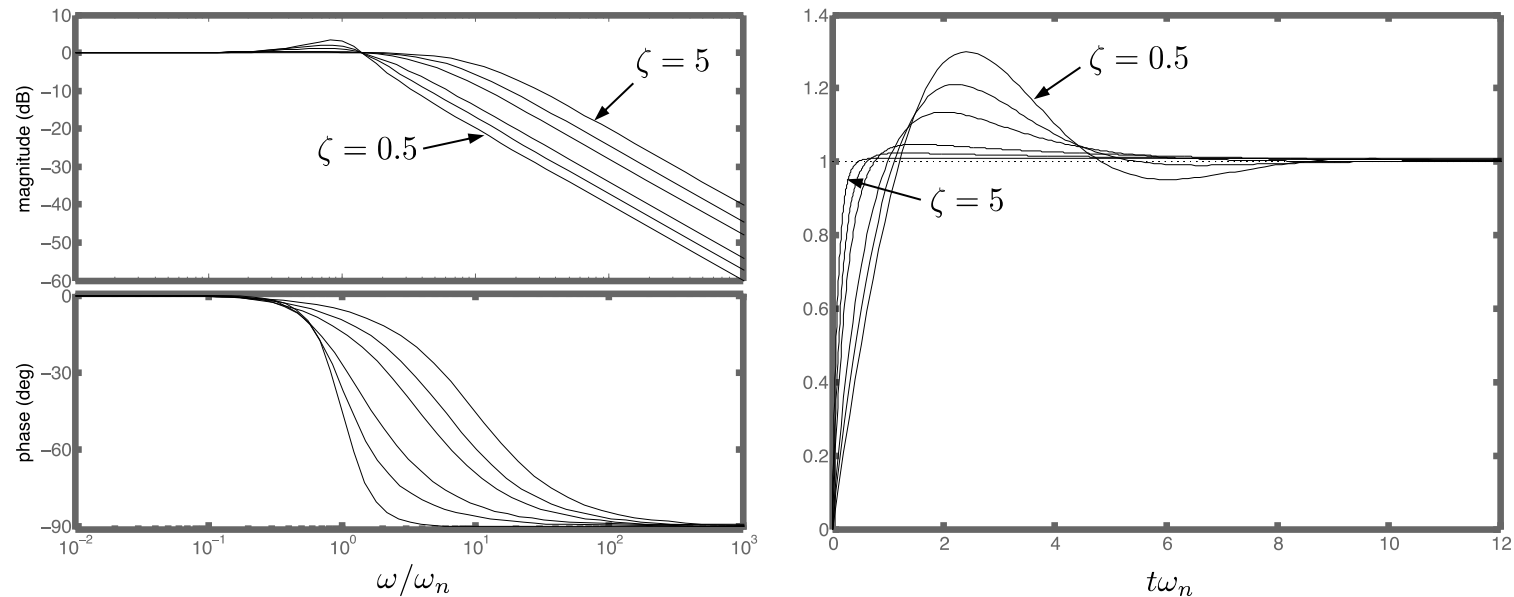
$$\chi = \frac{k_{p_\chi} g/V_a s + k_{i_\chi} g/V_a}{s^2 + k_{p_\chi} g/V_a s + k_{i_\chi} g/V_a} \chi^c + \frac{g/V_g s}{s^2 + k_{p_\chi} g/V_a s + k_{i_\chi} g/V_a} d_\chi$$

Заметим:

- В реакции на команду курса χ^c имеется нуль.
- Наличие нуля в начале координат обеспечивает подавление низкочастотных возмущений.



Нуль передаточной функции влияет на реакцию

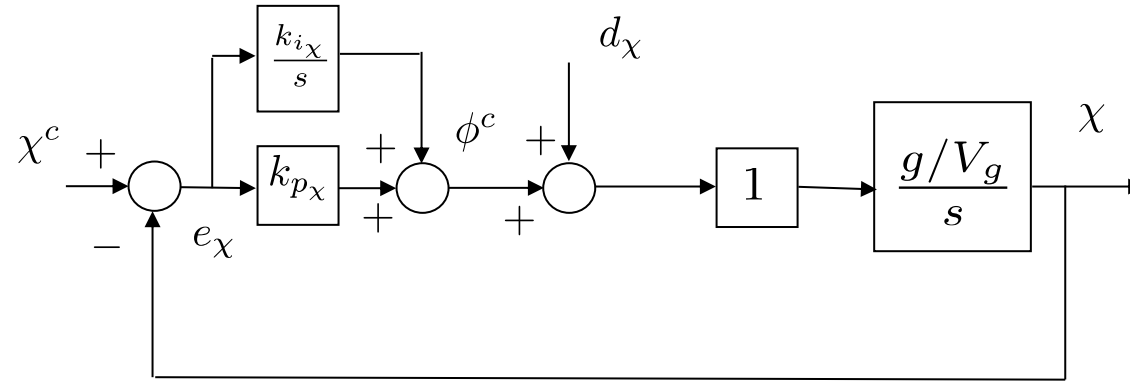


При наличии нуля каноническая передаточная функция второго порядка имеет вид

$$H(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} = \underbrace{\left(\frac{2\zeta}{\omega_n}\right) \frac{\omega_n^2 s}{s^2 + 2\zeta\omega_n s + \omega_n^2}}_{d/dt \text{ ПФ 2-го порядка}} + \underbrace{\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}}_{\text{ПФ 2-го порядка}}$$

Заметим, что ζ оказывает иное влияние при наличии нуля.

Контур удержания курса



$$\chi = \underbrace{\frac{(k_{p_\chi} g/V_g)s + (k_{i_\chi} g/V_g)}{s^2 + (k_{p_\chi} g/V_g)s + (k_{i_\chi} g/V_g)}}_{\text{Реакция на команду курса}} \chi^c + \underbrace{\frac{(g/V_g)s}{s^2 + (k_{p_\chi} g/V_g)s + (k_{i_\chi} g/V_g)}}_{\text{Реакция на возмущение}} d_\chi$$

Приравнивая коэффициенты к канонической ПФ, получаем:

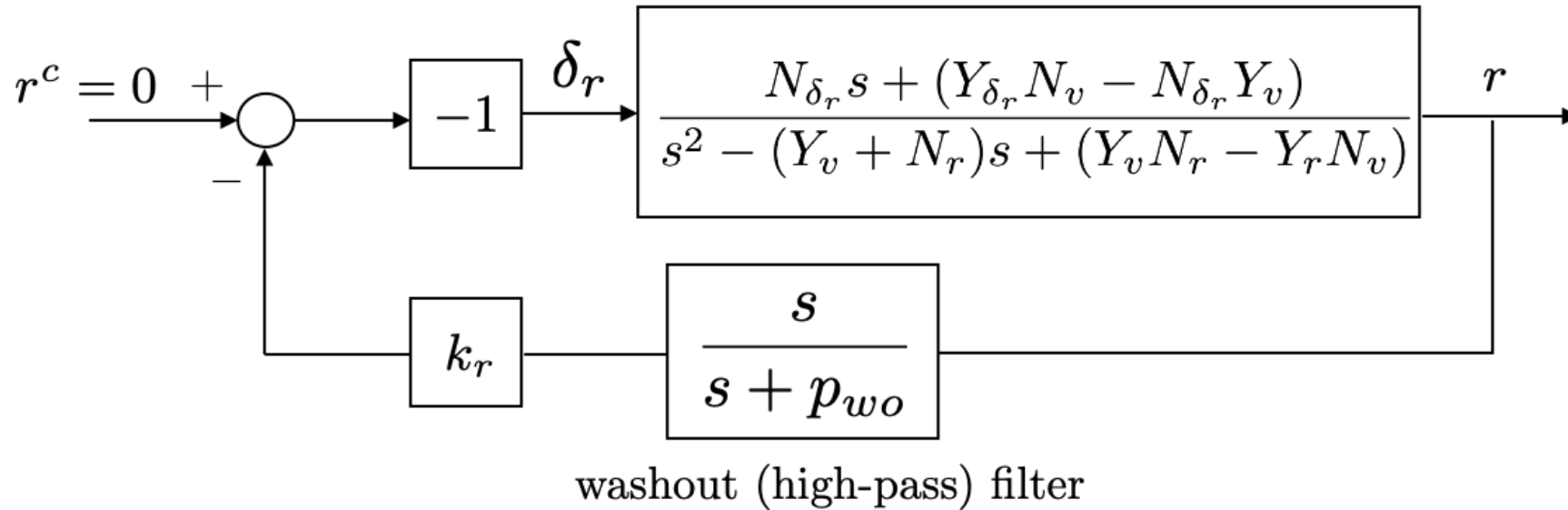
$$\omega_{n_\chi}^2 = k_{i_\chi} g/V_g \quad \text{и} \quad 2\zeta_\chi \omega_{n_\chi} = k_{p_\chi} g/V_g$$

или

$$\omega_{n_\chi} = \frac{1}{W_\chi} \omega_{n_\phi} \quad \boxed{k_{p_\chi} = 2\zeta_\chi \omega_{n_\chi} V_g/g} \quad \boxed{k_{i_\chi} = \omega_{n_\chi}^2 V_g/g}$$

Параметрами проектирования являются разнос полос пропускания W_χ и коэффициент демпфирования ζ_χ

Демпфер рыскания



- Руль направления используется для противодействия скорости рыскания, вызванной неблагоприятным рысканием
- Washout-фильтр обеспечивает то, что демпфер рыскания противодействует только высокочастотной скорости рыскания
- Washout-фильтр является фильтром верхних частот с частотой среза p_{wo}

Автопилот бокового движения - Итог

Если модель известна, параметрами проектирования являются:

Внутренний контур (удержание угла крена)

- ω_{n_ϕ} — ошибка по крену, когда элерон только достигает насыщения
- ζ_ϕ — коэффициент демпфирования контура стабилизации крена

Внешний контур (удержание курса)

- $W_\chi > 1$ — разнос полос пропускания между контурами крена и курса
- ζ_χ — коэффициент демпфирования контура удержания курса

Демпфер рыскания (если доступен руль направления)

- τ_r — частота среза washout-фильтра
- k_r — коэффициент усиления демпфера рыскания

Автопилот бокового движения – Настройка в полете

Если модель неизвестна и автопилот должен быть настроен в полёте, то следующие коэффициенты усиления настраиваются по одному за раз, в указанном порядке:

Внутренний контур (удержание угла крена)

- k_{d_ϕ} — Увеличивайте k_{d_ϕ} до появления неустойчивости, затем уменьшите на 20%
- k_{p_ϕ} — Настройте k_{p_ϕ} для получения приемлемой реакции на ступенчатое воздействие

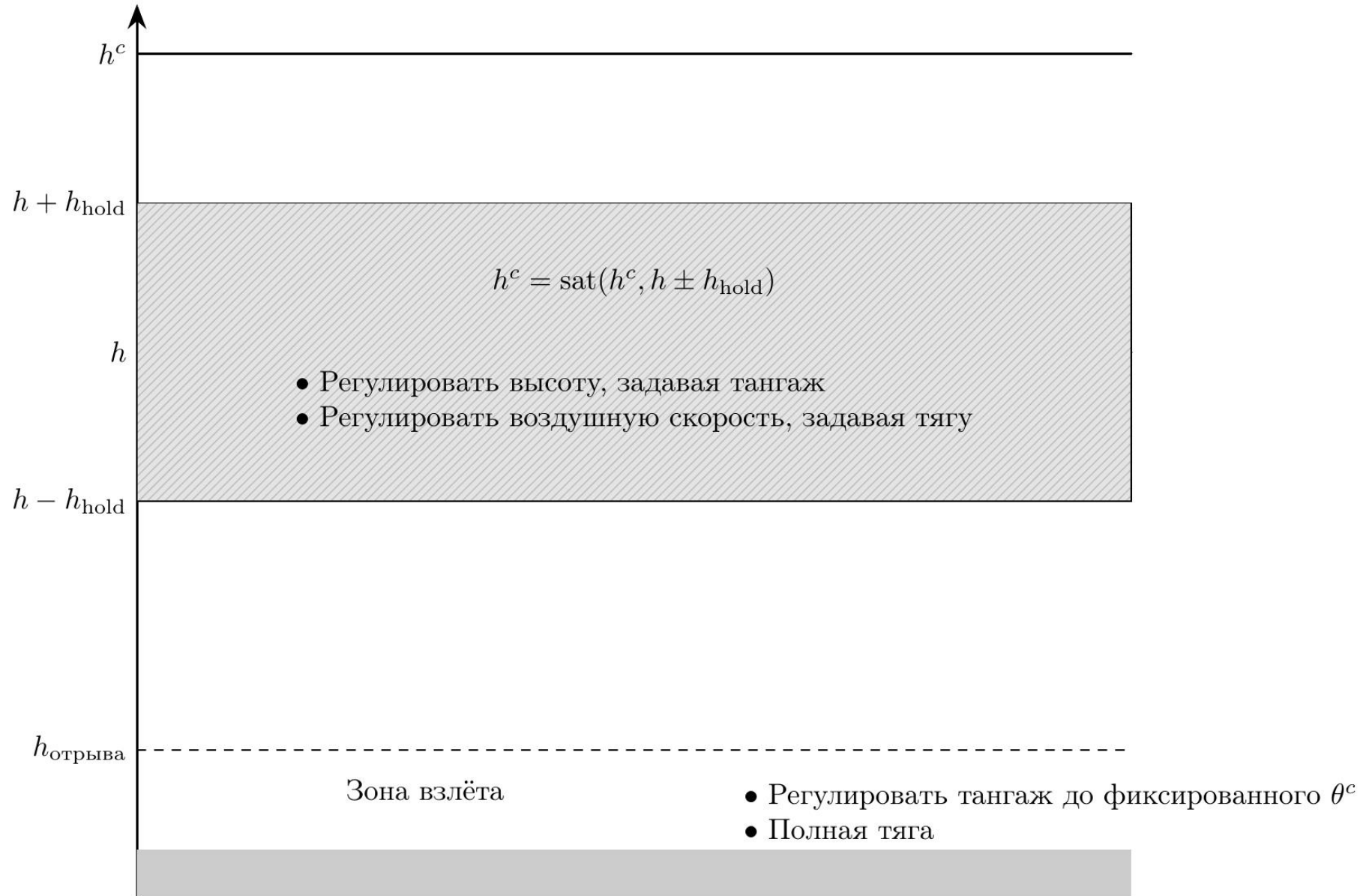
Внешний контур (удержание курса)

- k_{p_χ} — Настройте k_{p_χ} для получения приемлемой реакции на ступенчатое воздействие
- k_{i_χ} — Настройте k_{i_χ} для устранения установившейся ошибки

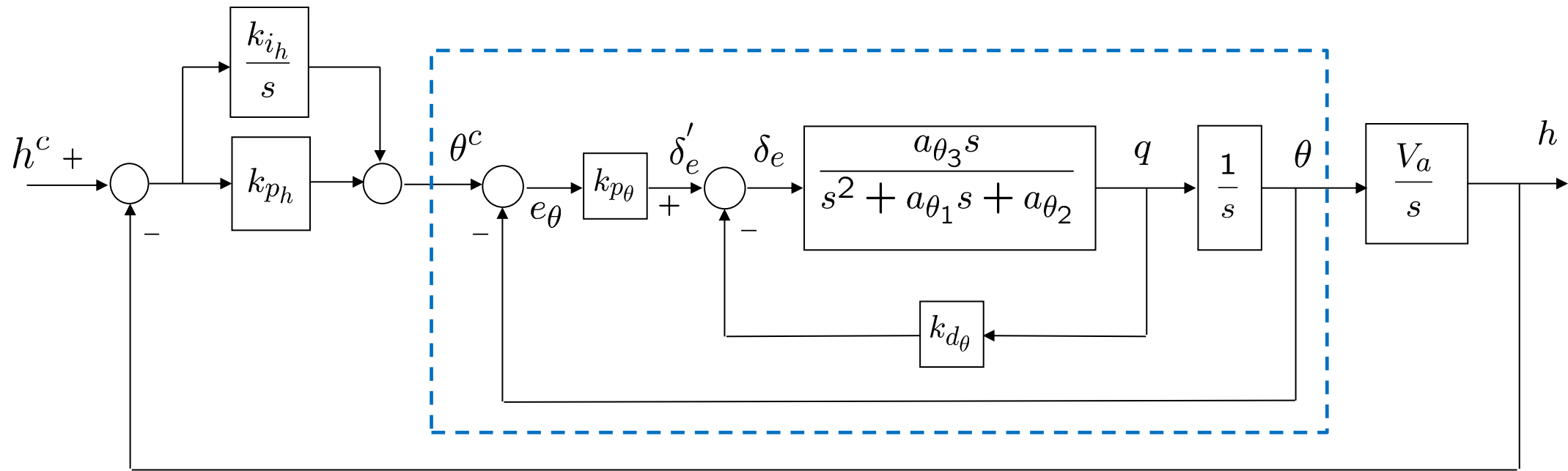
Удержание бокового скольжения (если доступен руль направления)

- τ_r — Настройте τ_r для получения приемлемой реакции на ступенчатое воздействие
- k_r — Настройте k_r для устранения установившейся ошибки

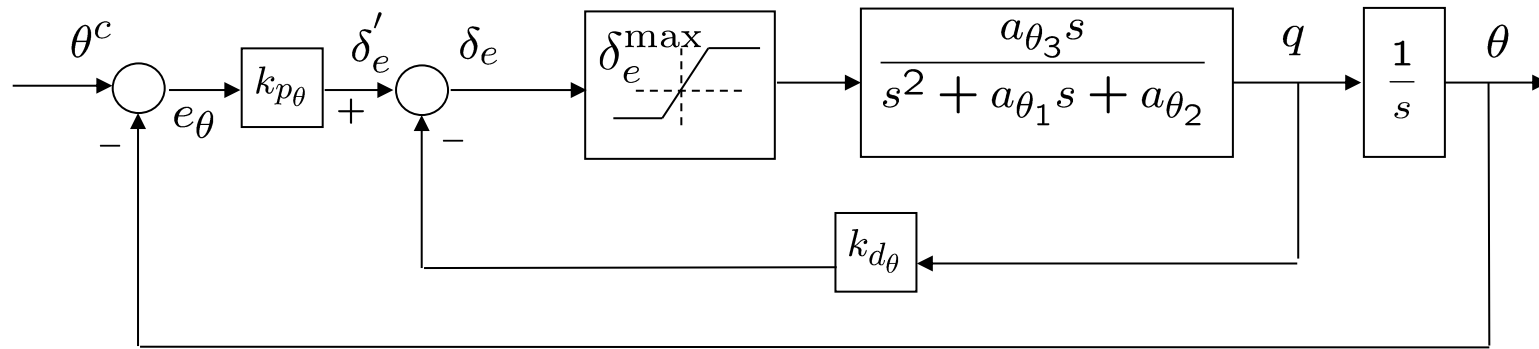
Продольные режимы полёта



Стабилизация высоты через управление тангажом



Удержание угла тангажа



$$H_{\theta/\theta^c}(s) = \frac{k_{p\theta} a_{\theta_3}}{\underbrace{s^2 + (a_{\theta_1} + k_{d\theta} a_{\theta_3})s + (a_{\theta_2} + k_{p\theta} a_{\theta_3})}_{\text{ПФ замкнутой системы}}} = \frac{K_{\theta\text{DC}} \omega_{n\theta}^2}{\underbrace{s^2 + 2\zeta_{\theta} \omega_{n\theta} s + \omega_{n\theta}^2}_{\text{Примечание: коэффициент усиления на постоянном сигнале не равен единице}}}$$

Приравнивая коэффициенты, получаем выражения для коэффициентов усиления:

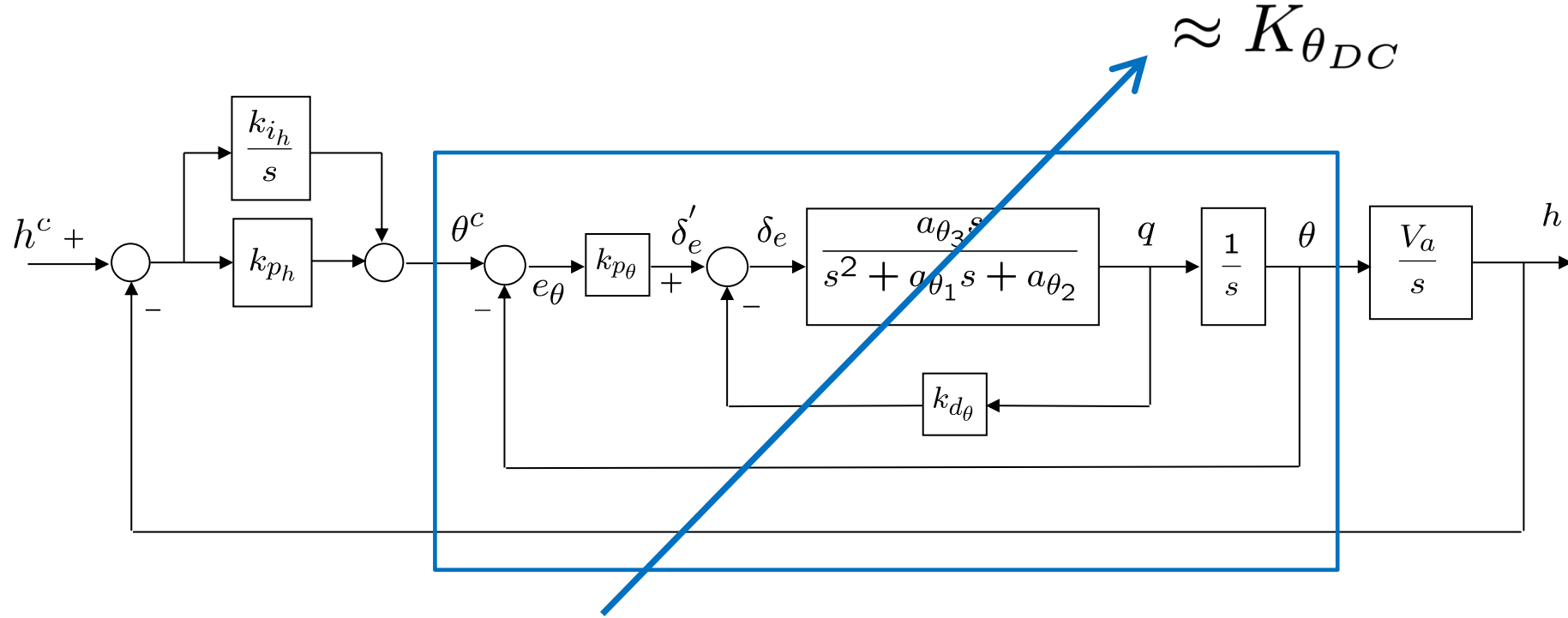
$$k_{p\theta} = \frac{\omega_{n\theta}^2 - a_{\theta_2}}{a_{\theta_3}}$$

$$k_{d\theta} = \frac{2\zeta_{\theta} \omega_{n\theta} - a_{\theta_1}}{a_{\theta_3}}$$

Параметрами проектирования являются $\omega_{n\theta}$ и ζ_{θ}
Коэффициент усиления на постоянном сигнале равен

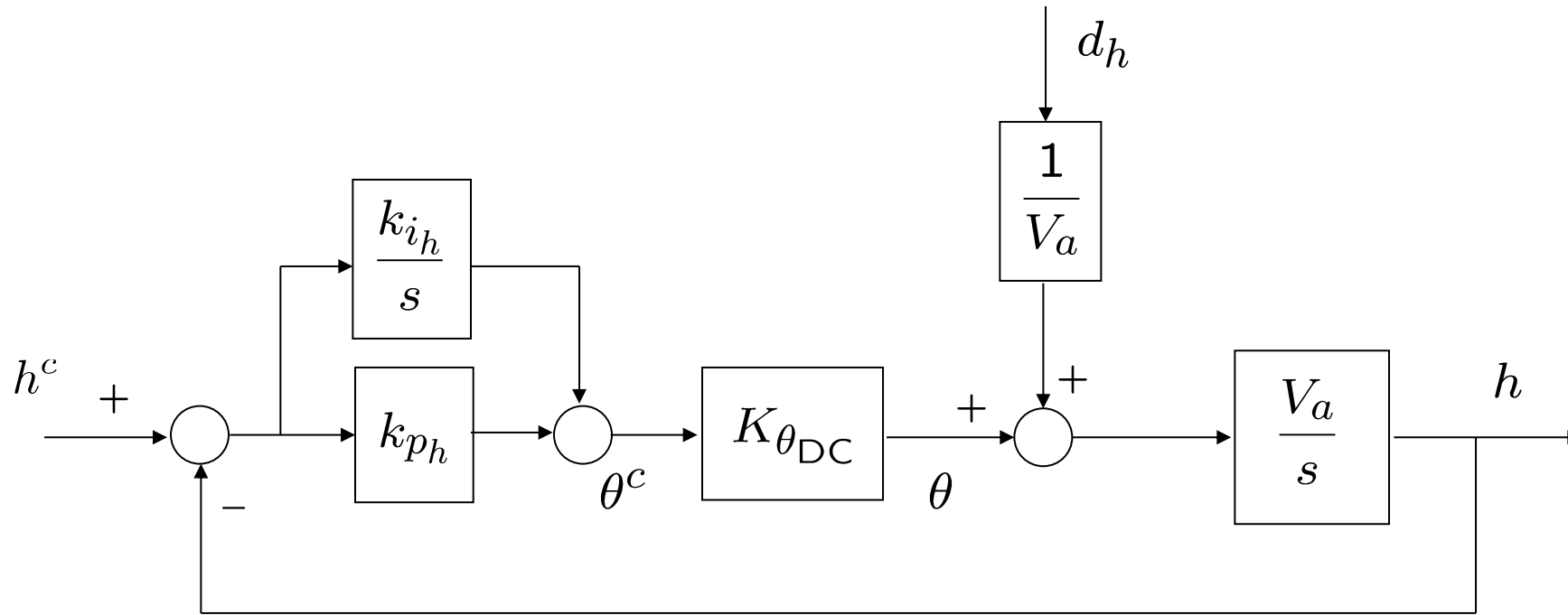
$$K_{\theta\text{DC}} = \frac{k_{p\theta} a_{\theta_3}}{a_{\theta_2} + k_{p\theta} a_{\theta_3}}$$

Стабилизация высоты через управление тангажом



При условии, что контур тангажа работает должным образом, мы можем упростить динамику внутреннего контура до $\theta \approx K_{\theta DC} \theta^c$

Высота через тангаж - упрощённо



$$H(s) = \left(\frac{K_{\theta_{\text{DC}}} V_a k_{ph} s + K_{\theta_{\text{DC}}} V_a k_{ih}}{s^2 + K_{\theta_{\text{DC}}} V_a k_{ph} s + K_{\theta_{\text{DC}}} V_a k_{ih}} \right) h^c(s) + \left(\frac{s}{s^2 + K_{\theta_{\text{DC}}} V_a k_{ph} s + K_{\theta_{\text{DC}}} V_a k_{ih}} \right) d_h(s)$$

ПИ-управление по высоте обеспечивает отслеживание постоянного задания h^c с нулевой установившейся ошибкой и подавление постоянных возмущений.

Расчёт коэффициентов усиления для высоты от тангажа

Приравнивая передаточные функции

$$H_{h/h^c}(s) = \frac{K_{\theta_{DC}} V_a k_{p_h} s + K_{\theta_{DC}} V_a k_{i_h}}{s^2 + K_{\theta_{DC}} V_a k_{p_h} s + K_{\theta_{DC}} V_a k_{i_h}} \quad \text{и} \quad H_c(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

получаем коэффициенты

$$k_{i_h} = \frac{\omega_{n_h}^2}{K_{\theta_{DC}} V_a}$$

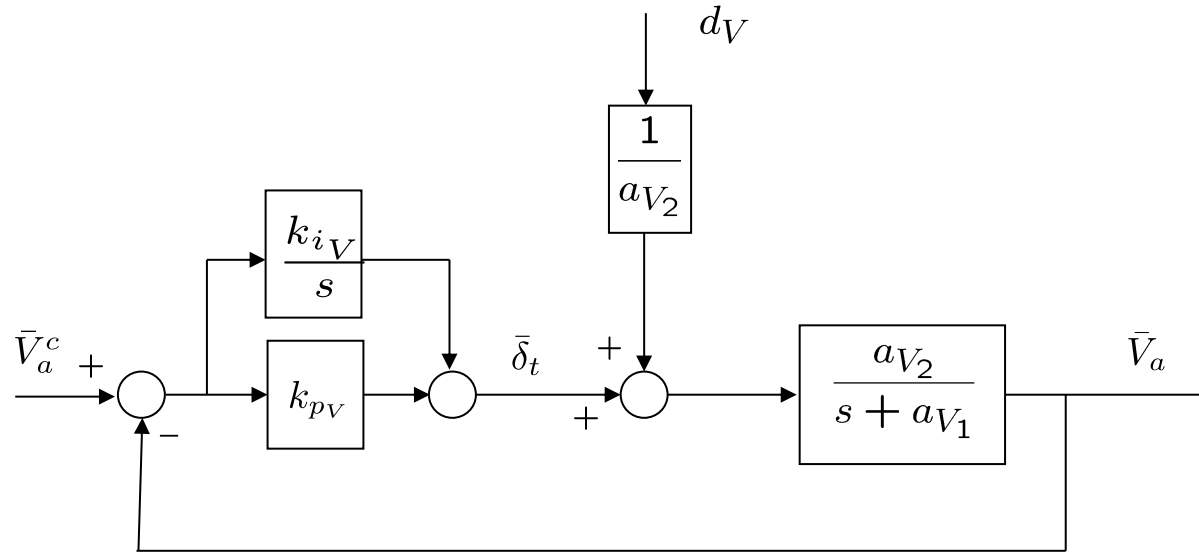
$$k_{p_h} = \frac{2\zeta_h \omega_{n_h}}{K_{\theta_{DC}} V_a}$$

где разнос полос пропускания достигается выбором

$$\omega_{n_h} = \frac{1}{W_h} \omega_{n_\theta}$$

Параметрами проектирования являются разнос полос пропускания W_h и коэффициент демпфирования ζ_θ

Стабилизация воздушной скорости через управление тягой



$$V_a = \left(\frac{a_{V_2}(k_{pV}s + k_{iV})}{s^2 + (a_{V_1} + a_{V_2}k_{pV})s + a_{V_2}k_{iV}} \right) V_a^c + \left(\frac{s}{s^2 + (a_{V_1} + a_{V_2}k_{pV})s + a_{V_2}k_{iV}} \right) d_V$$

ПИ-управление в контуре «тяга – воздушная скорость» обеспечивает отслеживание постоянного задания V_a^c с нулевой установившейся ошибкой и подавление постоянных возмущений.

Расчёт коэффициентов усиления в канале «воздушная скорость – тяга»

Приравнивая передаточные функции и их коэффициенты

$$H_{V_a/V_a^c}(s) = \left(\frac{a_{V_2} k_{pV} s + a_{V_2} k_{iV}}{s^2 + (a_{V_1} + a_{V_2} k_{pV})s + a_{V_2} k_{iV}} \right) = \frac{\rho 2\zeta \omega_n s + \omega_n^2}{s^2 + 2\zeta \omega_n s + \omega_n^2}$$

получаем коэффициенты усиления

$$k_{iV} = \frac{\omega_{nV}^2}{a_{V_2}}$$

$$k_{pV} = \frac{2\zeta_V \omega_{nV} - a_{V_1}}{a_{V_2}}$$

Параметрами проектирования являются собственная частота ω_{nV} и коэффициент демпфирования ζ_V

Управляющий сигнал имеет вид

$$\begin{aligned} \delta_t &= \delta_t^* + \bar{\delta}_t \\ &= \delta_t^* + k_{pV} (V_a^c - V_a) + k_{iV} \int (V_a^c - V_a) dt \end{aligned}$$

Заметим, что если δ_t^* известно неточно, интегратор это скомпенсирует.

Автопилот продольного движения - Итоги

Если модель известна, параметрами проектирования являются:

Внутренний контур (удержание угла тангажа)

- ω_{n_θ} — собственная частота тангажа.
- ζ_θ — коэффициент демпфирования контура удержания угла тангажа.

Внешний контур удержания высоты

- $W_h > 1$ — разнос полос пропускания между контурами тангажа и высоты.
- ζ_h — коэффициент демпфирования контура удержания высоты.

Внешний контур удержания воздушной скорости

- $W_{V_2} > 1$ — разнос полос пропускания между контурами тангажа и воздушной скорости.
- ζ_{V_2} — коэффициент демпфирования контура удержания воздушной скорости.

Контур удержания тяги (внутренний контур)

- ω_{n_V} — собственная частота контура тяги.
- ζ_V — коэффициент демпфирования контура тяги.

Автопилот продольного движения – Настройка

Если модель неизвестна и автопилот должен быть настроен в полёте, то следующие коэффициенты усиления настраиваются по одному за раз, в указанном порядке:

Внутренний контур (удержание угла тангажа)

- k_{d_θ} — Увеличивайте k_{d_θ} до появления неустойчивости, затем уменьшите на 20%.
- k_{p_θ} — Настройте k_{p_θ} для получения приемлемой реакции на ступенчатое воздействие.

Внешний контур удержания высоты

- k_{p_h} — Настройте k_{p_h} для получения приемлемой реакции на ступенчатое воздействие.
- k_{i_h} — Настройте k_{i_h} для устранения установившейся ошибки.

Внешний контур удержания воздушной скорости

- $k_{p_{V_2}}$ — Настройте $k_{p_{V_2}}$ для получения приемлемой реакции на ступенчатое воздействие.
- $k_{i_{V_2}}$ — Настройте $k_{i_{V_2}}$ для устранения установившейся ошибки.

Контур удержания тяги (внутренний контур)

- k_{p_V} — Настройте k_{p_V} для получения приемлемой реакции на ступенчатое воздействие.
- k_{i_V} — Настройте k_{i_V} для устранения установившейся ошибки.

Реализация в Python

```
class autopilot:
    def __init__(self, ts_control):
        self.roll_from_aileron = pdControlWithRate(kp, kd, limit)
        self.course_from_roll = piControl(kp, ki, Ts, limit)
        self.yaw_damper = transferFunction(num, den, Ts)
        self.pitch_from_elevator = pdControlWithRate(kp, kd, limit)
        self.altitude_from_pitch = piControl(kp, ki, Ts, limit)
        self.airspeed_from_throttle = piControl(kp, ki, Ts, limit)

    def update(self, cmd, state):
        # lateral autopilot
        chi_c = wrap(cmd.course_command, state.chi)
        phi_c = self.saturate(
            cmd.phi_feedforward + self.course_from_roll.update(chi_c, state.chi)
            - np.radians(30), np.radians(30))
        delta_a = self.roll_from_aileron.update(phi_c, state.phi, state.p)
        delta_r = self.yaw_damper.update(state.r)

        # longitudinal autopilot
        # saturate the altitude command
        h_c = self.saturate(cmd.altitude_command,
                           state.h - AP.altitude_zone, state.h + AP.altitude_zone)
        theta_c = self.altitude_from_pitch.update(h_c, state.h)
        delta_e = self.pitch_from_elevator.update(theta_c, state.theta, state.q)
        delta_t = self.airspeed_from_throttle.update(cmd.airspeed_command,
                                                    state.Va)

        delta_t = self.saturate(delta_t, 0.0, 1.0)

        return delta, self.commanded_state
```

Функция нормализации угла (Wrap Function)

```
import numpy as np

def wrap( chi_1 ,  chi_2 ):
    while chi_1 - chi_2 > np.pi:
        chi_1 = chi_1 - 2.0 * np.pi
    while chi_1 - chi_2 < -np.pi:
        chi_1 = chi_1 + 2.0 * np.pi
    return chi_1
```

Реализация контура с ПИД-регулятором

$$u(t) = k_p e(t) + k_i \int_{-\infty}^t e(\tau) d\tau + k_d \frac{de}{dt}(t) \quad \text{ПИД-регулятор в непрерывном времени}$$

$$e(t) = y^c(t) - y(t)$$

Применяя преобразование Лапласа...

$$U(s) = k_p E(s) + k_i \frac{E(s)}{s} + k_d s E(s)$$

Использовать дифференциатор с ограниченной полосой для снижения шума

$$U(s) = k_p E(s) + k_i \frac{E(s)}{s} + k_d \frac{s}{\tau s + 1} E(s)$$

Реализация контура с ПИД-регулятором

Правило Тустина или метод трапеций:

$$s \mapsto \frac{2}{T_s} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)$$

Интегратор:

$$I(z) = \frac{T_s}{2} \left(\frac{1 + z^{-1}}{1 - z^{-1}} \right) E(z) \quad \Longrightarrow \quad \boxed{I[n] = I[n - 1] + \frac{T_s}{2} (E[n] + E[n - 1])}$$

Дифференциатор:

$$D(z) = \frac{\frac{2}{T_s} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)}{\frac{2\tau}{T_s} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) + 1} E(z) = \frac{\left(\frac{2}{2\tau + T_s} \right) (1 - z^{-1})}{1 - \left(\frac{2\tau - T_s}{2\tau + T_s} \right) z^{-1}} E(z)$$
$$\Longrightarrow \quad \boxed{D[n] = \left(\frac{2\tau - T_s}{2\tau + T_s} \right) D[n - 1] + \left(\frac{2}{2\tau + T_s} \right) (E[n] - E[n - 1])}$$

Защита интегратора от насыщения

$$u_{\text{unsat}}^- = k_p e + k_d D + k_i I^- \quad \text{управление до обновления анти-виндап}$$

$$u_{\text{unsat}}^+ = k_p e + k_d D + k_i I^+ \quad \text{управление после обновления анти-виндап}$$

$$I^+ = I^- + \Delta I \quad \Delta I \text{ — это поправка анти-виндап}$$

$$u_{\text{unsat}}^+ = u_{\text{unsat}}^- + k_i \Delta I \quad \text{вычитая два верхних уравнения}$$

Пусть $u_{\text{unsat}}^+ = u$ значение управления после применения насыщения

$$\Delta I = \frac{1}{k_i} (u - u_{\text{unsat}}^-) \quad \text{решая...}$$

$$I^+ = I^- + \frac{1}{k_i} (u - u_{\text{unsat}}^-)$$

ПИД-регулятор (Python)

```
class PIDControl:
    def __init__(self, kp=0.0, ki=0.0, kd=0.0, Ts=0.01, sigma=0.05, limit=1.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.Ts = Ts
        self.limit = limit
        self.integrator = 0.0
        self.error_delay_1 = 0.0
        self.error_dot_delay_1 = 0.0
        self.y_dot = 0.0
        self.y_delay_1 = 0.0
        self.y_dot_delay_1 = 0.0
        # gains for differentiator
        self.a1 = (2.0 * sigma - Ts) / (2.0 * sigma + Ts)
        self.a2 = 2.0 / (2.0 * sigma + Ts)

    def update(self, y_ref, y, reset_flag=False):
        if reset_flag is True:
            self.integrator = 0.0
            self.error_delay_1 = 0.0
            self.y_dot = 0.0
            self.y_delay_1 = 0.0
            self.y_dot_delay_1 = 0.0
        # compute the error
        error = y_ref - y
        # update the integrator using trapazoidal rule
        self.integrator = self.integrator \
            + (self.Ts/2) * (error + self.error_delay_1)
        # update the differentiator
        error_dot = self.a1 * self.error_dot_delay_1 \
            + self.a2 * (error - self.error_delay_1)
        # PID control
        u = self.kp * error \
            + self.ki * self.integrator \
            + self.kd * error_dot
        # saturate PID control at limit
        u_sat = self._saturate(u)
        # integral anti-windup
        # adjust integrator to keep u out of saturation
        if np.abs(self.ki) > 0.0001:
            self.integrator = self.integrator \
                + (self.Ts / self.ki) * (u_sat - u)
        # update the delayed variables
        self.error_delay_1 = error
        self.error_dot_delay_1 = error_dot
```

Интегратор:

$$I[n] = I[n-1] + \frac{T_s}{2}(E[n] + E[n-1])$$

Дифференциатор:

$$D[n] = \left(\frac{2\tau - T_s}{2\tau + T_s} \right) D[n-1] + \left(\frac{2}{2\tau + T_s} \right) (E[n] - E[n-1])$$

Защита интегратора от насыщения:

$$I^+ = I^- + \frac{1}{k_i}(u - u_{\text{unsat}}^-)$$

ПИД-регулятор (Matlab)

```
1 function u = pidloop(y_c, y, flag, kp, ki, kd, limit, Ts, tau)
2 persistent integrator;
3 persistent differentiator;
4 persistent error_d1;
5 if flag==1, % reset (initialize) persistent variables
6     % when flag==1
7     integrator = 0;
8     differentiator = 0;
9     error_d1 = 0; % _d1 means delayed by one time step
10 end
11 error = y_c - y; % compute the current error
12 integrator = integrator + (Ts/2)*(error + error_d1);
13 % update integrator
14 differentiator = (2*tau-Ts)/(2*tau+Ts)*differentiator...
15     + 2/(2*tau+Ts)*(error - error_d1);
16 % update differentiator
17 error_d1 = error; % update the error for next time through
18     % the loop
19 u = sat(... % implement PID control
20     kp * error + ... % proportional term
21     ki * integrator + ... % integral term
22     kd * differentiator, ... % derivative term
23     limit... % ensure abs(u)<=limit
24 );
25 % implement integrator anti-windup
26 if ki~=0
27     u_unsat = kp*error + ki*integrator + kd*differentiator;
28     integrator = integrator + Ts/ki * (u - u_unsat);
29 end
30
31 function out = sat(in, limit)
32 if in > limit, out = limit;
33 elseif in < -limit; out = -limit;
34 else out = in;
35 end
```

Интегратор:

$$I[n] = I[n - 1] + \frac{T_s}{2}(E[n] + E[n - 1])$$

Дифференциатор:

$$D[n] = \left(\frac{2\tau - T_s}{2\tau + T_s} \right) D[n - 1] + \left(\frac{2}{2\tau + T_s} \right) (E[n] - E[n - 1])$$

Защита интегратора от насыщения:

$$I^+ = I^- + \frac{1}{k_i}(u - u_{\text{unsat}}^-)$$

Реализация демпфера рыскания

Для передаточной функции первого порядка в непрерывном времени вида

$$H(s) = \frac{U(s)}{Y(s)} = k \left(\frac{n_1 s + n_0}{d_1 s + d_0} \right)$$

мы хотим иметь возможность вычислить эквивалентную передаточную функцию в дискретном времени

$$H(z) = \frac{U(z)}{Y(z)} = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}$$

для заданной частоты дискретизации T_s . Используя метод Тастина (метод трапеций) подстановкой

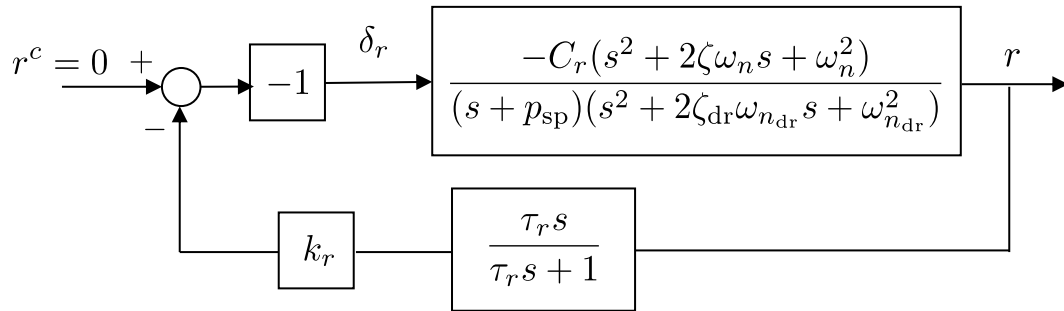
$s \rightarrow \frac{2}{T_s} \left(\frac{1-z^{-1}}{1+z^{-1}} \right)$, мы можем вычислить коэффициенты $H(z)$ как

$$\begin{aligned} b_0 &= \frac{k(2n_1 + T_s n_0)}{2d_1 + T_s d_0} & b_1 &= -\frac{k(2n_1 - T_s n_0)}{2d_1 + T_s d_0} \\ a_1 &= -\frac{k(2d_1 - T_s d_0)}{2d_1 + T_s d_0} \end{aligned}$$

Взяв обратное z -преобразование, мы можем преобразовать нашу дискретную передаточную функцию в разностное уравнение дискретного времени:

$$u_k = -a_1 u_{k-1} + b_0 y_k + b_1 y_{k-1}$$

Реализация демпфера рыскания



Для нашего демпфера рыскания,

$$H(s) = \frac{\delta_r(s)}{r(s)} = k_r \left(\frac{s}{\tau_r s + 1} \right) = k_r \left(\frac{s}{s + p_{wo}} \right),$$

где $k_r = 0.2$, $p_{wo} = 0.45$ рад/с, а $T_s = 0.01$ с

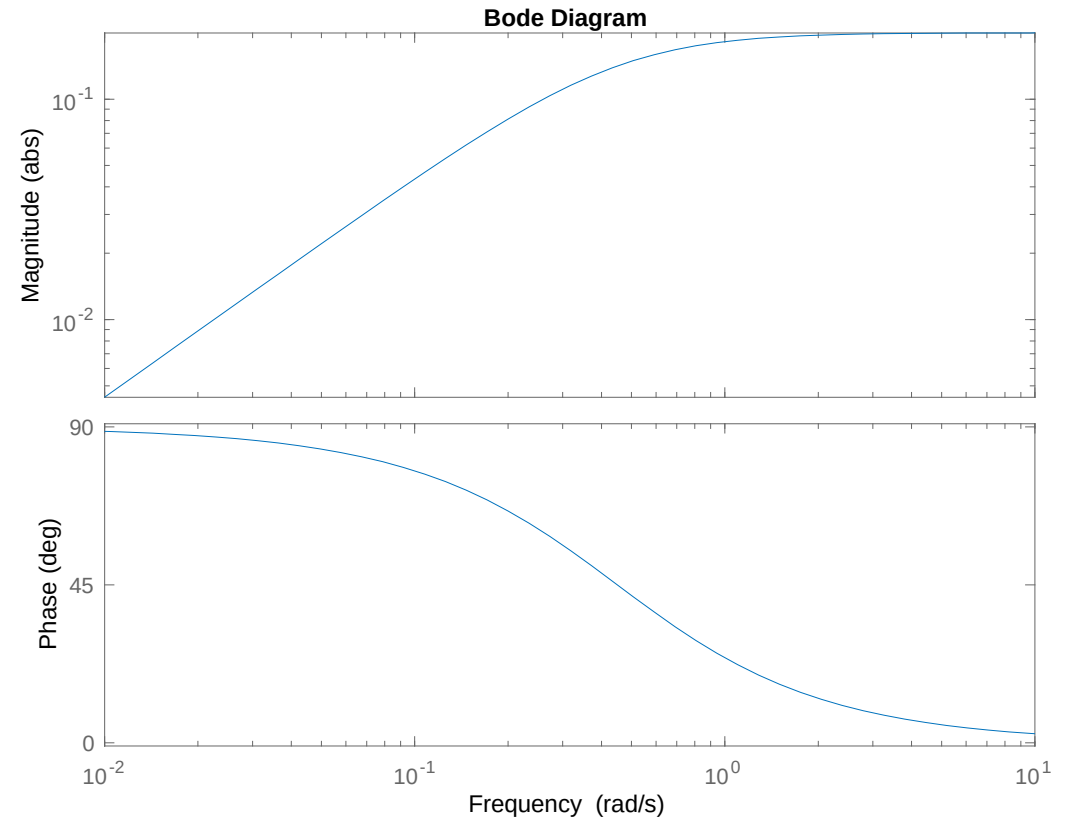
Это даёт передаточную функцию в дискретном времени

$$H(z) = \frac{\delta_r(z)}{r(z)} = \frac{0.1996 - 0.1996 z^{-1}}{1 - 0.9955 z^{-1}}$$

с соответствующим разностным уравнением

$$\delta_{r,k} = 0.9955 \delta_{r,k-1} + 0.1996 r_k - 0.1996 r_{k-1}$$

Коэффициенты $H(z)$ можно вычислить вручную, как мы сделали здесь, или их можно вычислить в MATLAB с помощью команды `c2d` с опцией `'tustin'` или в Python с помощью функции `control.matlab.c2d`. Это особенно удобно для передаточных функций более высокого порядка.

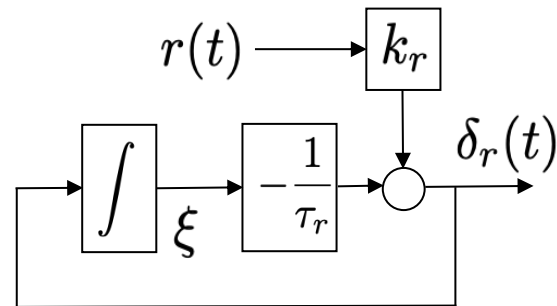


Реализация демпфера рыскания

Демпфер рыскания: $\delta_r(s) = k_r \left(\frac{s}{s + \frac{1}{\tau_r}} \right) r(s)$

Во временной области: $\dot{\delta}_r = -\frac{1}{\tau_r} \delta_r + k_r \dot{r}$

Интегрируя: $\delta_r(t) = -\frac{1}{\tau_r} \int_{-\infty}^t \delta_r(\sigma) d\sigma + k_r r(t)$



Форма в пространстве состояний: Дискретное время:

$$\begin{aligned} \dot{\xi} &= -\frac{1}{\tau_r} \xi + k_r r & \xi_k &= \xi_{k-1} + T_s \left(-\frac{1}{\tau_r} \xi_{k-1} + k_r r_{k-1} \right) \\ \delta_r &= -\frac{1}{\tau_r} \xi + k_r r. & \delta_{r_k} &= -\frac{1}{\tau_r} \xi_k + k_r r_k. \end{aligned}$$

Реализация демпфера рыскания – Альтернативная

Дискретное время:

$$\xi_k = \xi_{k-1} + T_s \left(-\frac{1}{\tau_r} \xi_{k-1} + k_r r_{k-1} \right)$$
$$\delta_{r_k} = -\frac{1}{\tau_r} \xi_k + k_r r_k.$$

```
class yawDamper:
    def __init__(self, k_r, tau_r, Ts):
        # set initial conditions
        self.xi = 0.
        self.Ts = Ts
        self.k_r = k_r
        self.tau_r = tau_r

    def update(self, u):
        self.xi = self.xi
            + self.Ts * (-1/self.tau_r * self.xi + self.k_r * r)
        delta_r = -1/self.tau_r * self.xi + self.k_r * r
        return delta_r
```

Проект для моделирования

Шаг 1. Контур крена. Для модели Aerosonde используйте $V_a = 25$ м/с. Приведите летательный аппарат к балансировочному режиму и настройте реакцию на ступенчатое воздействие по крену.

Шаг 2. Настройте демпфер рыскания так, чтобы поддерживать $\beta \approx 0$.

Шаг 3. Контур курса. Настройте реакцию на ступенчатое воздействие по χ .

Шаг 4. Контур тангажа. Настройте реакцию на ступенчатое воздействие по углу тангажа.

Шаг 5. Настройте реакцию на ступенчатое воздействие по воздушной скорости.

Шаг 6. Настройте реакцию на ступенчатое воздействие по высоте (после насыщения).

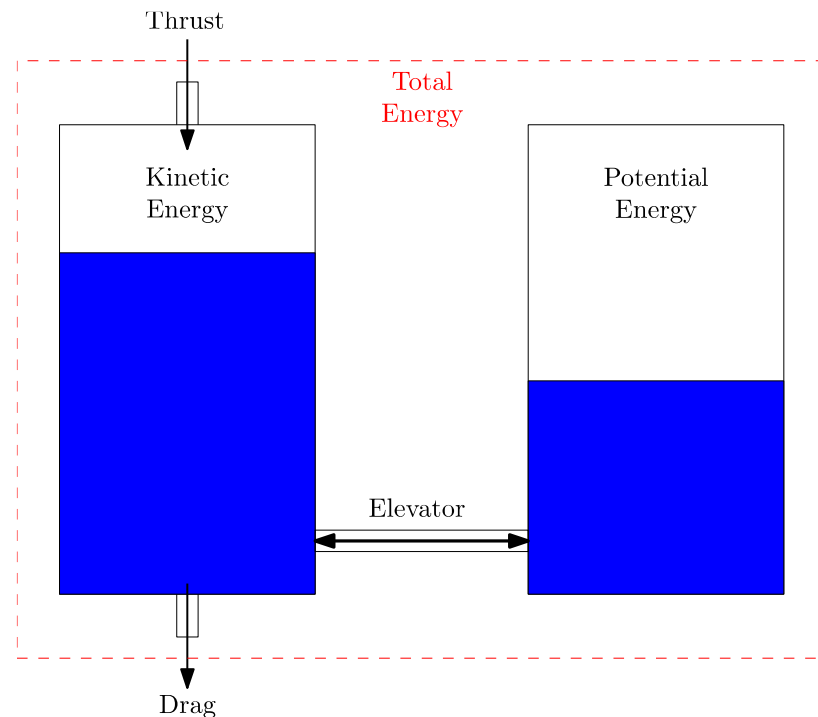
Шаг 7. Стресс-тест моделирования со взлёта. Подавайте ступенчатые команды по высоте, воздушной скорости, курсу.

Итог

- Последовательное замыкание контуров
- **Управление полной энергией**
- LQR-управление

Управление полной энергией (TECS)

- Разработано в 1980-х годах Антониусом Ламбрегтсом
- Основано на методах управления энергией 1950-х годов
- Управлять энергией системы вместо высоты и воздушной скорости



Управление полной энергией (TECS)

Кинетическая энергия: $E_K \triangleq \frac{1}{2}mV_a^2$

Потенциальная энергия: $E_P \triangleq mgh$

Полная энергия: $E_T \triangleq E_P + E_K$

Разность энергий: $E_D \triangleq E_P - E_K$

Определение ошибок:

$$\tilde{E}_K = \frac{1}{2}m \left((V_a^d)^2 - V_a^2 \right)$$

$$\tilde{E}_P = mg (h^d - h)$$

$$\tilde{E}_T = \tilde{E}_P + \tilde{E}_K$$

$$\tilde{E}_D = \tilde{E}_P - \tilde{E}_K$$

Управление полной энергией (TECS)

Оригинальный метод TECS, предложенная Ламбрегтсом, основана на скоростях изменения энергии:

- $T^c = T_D + k_{p,t} \dot{E}_t + k_{i,t} \int_{t_0}^t \dot{E}_t \delta \tau$
 - T_D — тяга, необходимая для компенсации сопротивления
 - ПИ-регулятор на основе скорости изменения полной энергии
- $\theta^c = k_{p,\theta} \dot{E}_d + k_{i,\theta} \int_{t_0}^t \dot{E}_d \delta \tau$
 - ПИ-регулятор на основе скорости изменения распределения энергии
- Устойчивость показана для линейных систем

Мы покажем, что эффективность этой схемы оставляет желать лучшего.

Управление полной энергией (TECS)

Если тяга, необходимая для компенсации сопротивления, неизвестна, то одна из возможностей — использовать интегратор для нахождения T_D :

- $T^c = k_{p,t}\tilde{E}_t + k_{i,t} \int \tilde{E}_t d\tau + k_{d,t}\dot{\tilde{E}}_t$
 - ПИД-регулятор на основе полной энергии (не скорости изменения энергии)
- $\theta^c = k_{p,\theta}\tilde{E}_d + k_{i,\theta} \int \tilde{E}_d d\tau + k_{d,\theta}\dot{\tilde{E}}_d$
 - ПИД-регулятор на основе распределения энергии (не скорости изменения)

Управление полной энергией (TECS)

Нелинейный повторный вывод:

- Определения ошибок

$$\tilde{E}_K = \frac{1}{2}m \left((V_a^d)^2 - V_a^2 \right)$$

$$\tilde{E}_P = mg (h^d - h)$$

- Функция Ляпунова

$$V = \frac{1}{2}\tilde{E}_T^2 + \frac{1}{2}\tilde{E}_D^2$$

- Регулятор

$$T^c = D + \frac{\tilde{E}_T^d}{V_a} + k_T \frac{\tilde{E}_T}{V_a}$$

$$\gamma^c = \sin^{-1} \left(\frac{\dot{h}^d}{V_a} + \frac{1}{2mgV_a} \left(-k_1 \tilde{E}_K + k_2 \tilde{E}_P \right) \right)$$

Управление полной энергией (TECS)

- Оригинал:
$$T^c = D + k_{p,t} \frac{\dot{E}_T}{mgV_a} + k_{i,t} \frac{\tilde{E}_T}{mgV_a}$$
- Нелинейный:
$$T^c = D + \frac{\dot{E}_T^d}{V_a} + k_T \frac{\tilde{E}_T}{V_a}$$

Аналогично, если $k_{p,T} = mg$ и $k_{i,T} = mgk_T$.

Нелинейный регулятор использует желаемую скорость изменения энергии.

Управление полной энергией (TECS)

- Модифицированный оригинал (Ardupilot):

$$\theta^c = \frac{k_{p,\theta}}{V_a m g} \left((2 - k) \dot{E}_P - k \dot{E}_K \right) + \frac{k_{i,\theta}}{V_a m g} \tilde{E}_D$$
$$k \in [0, 2]$$

- Нелинейный:

$$\gamma^c = \sin^{-1} \left(\frac{\dot{h}^d}{V_a} + \frac{1}{2mgV_a} \left(-k_1 \tilde{E}_K + k_2 \tilde{E}_P \right) \right)$$
$$k_1 \triangleq |k_T - k_D|$$
$$k_2 \triangleq k_T + k_D$$
$$0 < k_T \leq k_D$$

Вывод по Ляпунову предполагает, что ошибка потенциальной энергии должна иметь больший вес, чем кинетическая энергия

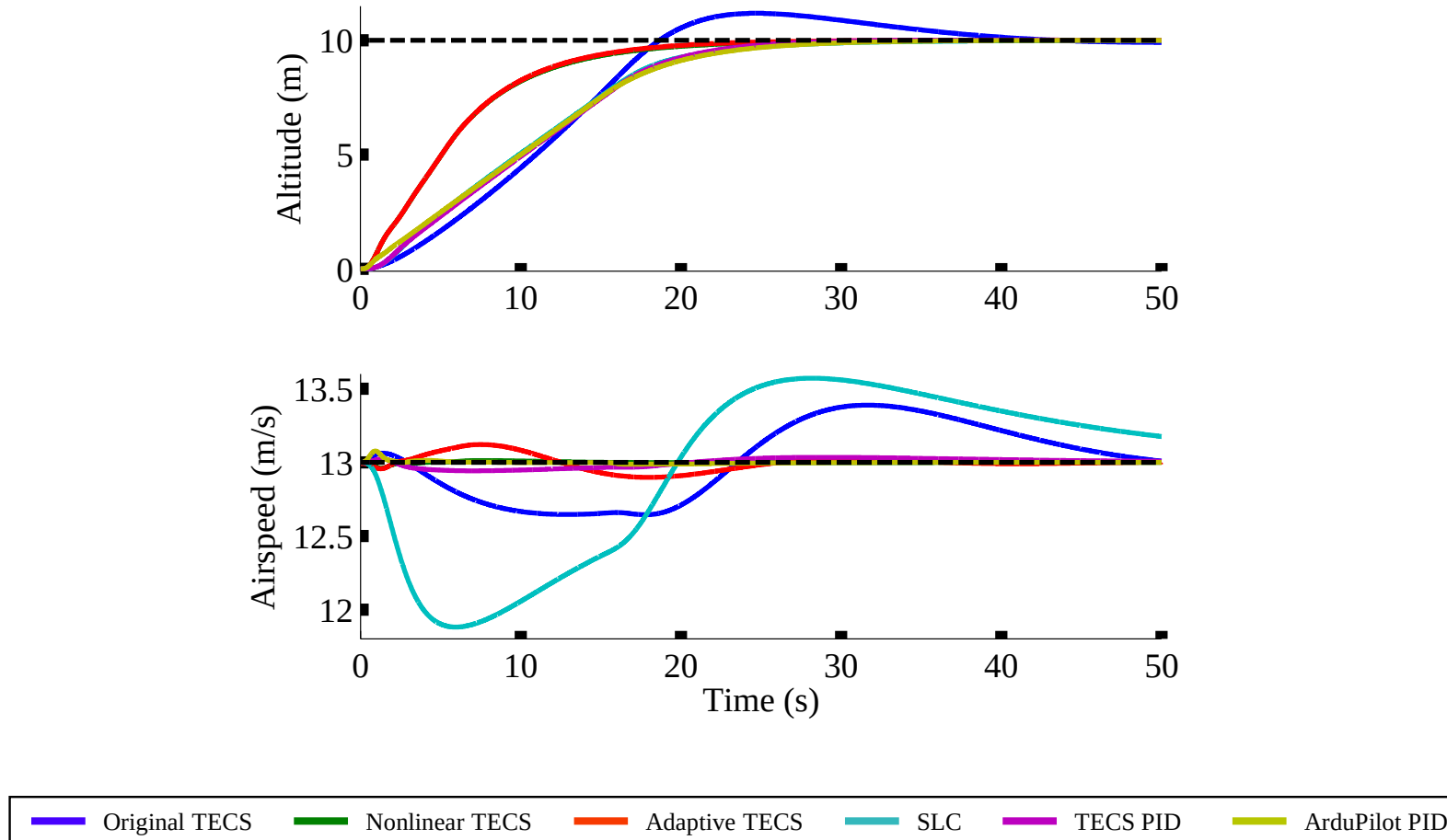
Управление полной энергией (TECS)

Если сопротивление неизвестно, то мы можем добавить адаптивную оценку:

$$\begin{aligned}T^c &= \hat{D} + \mathbf{\Phi}^\top \hat{\mathbf{\Psi}} + \frac{\dot{E}_T^d}{V_a} + k_T \frac{\tilde{E}_T}{V_a} \\ \gamma^c &= \sin^{-1} \left(\frac{\dot{h}^d}{V_a} + \frac{1}{2mgV_a} \left(-k_1 \tilde{E}_K + k_2 \tilde{E}_P \right) \right) \\ \dot{\hat{\mathbf{\Psi}}} &= \left(\Gamma_T \tilde{E}_T - \Gamma_D \tilde{E}_D \right) \mathbf{\Phi} V_a\end{aligned}$$

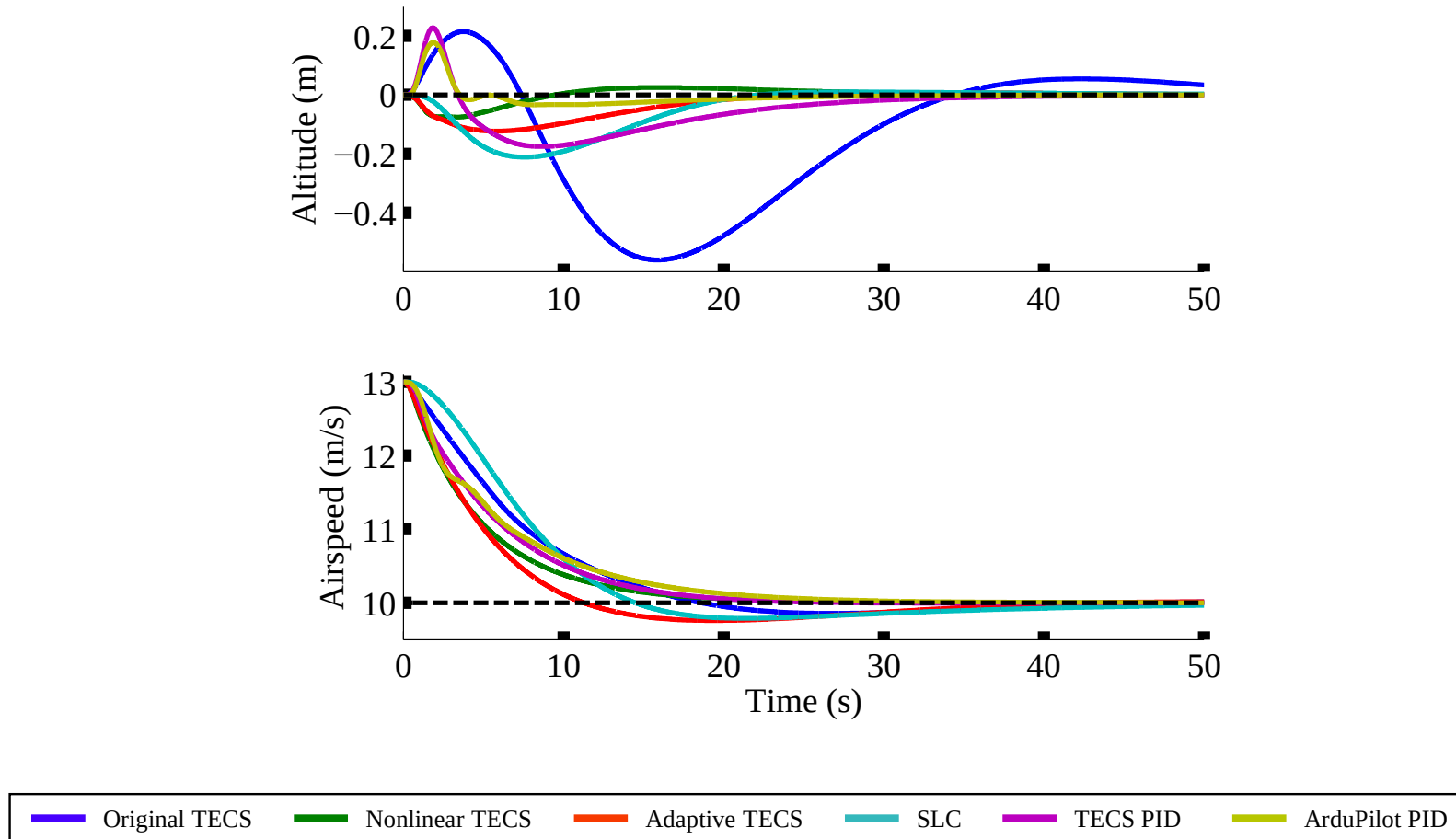
Управление полной энергией (TECS)

Ступенчатое воздействие по высоте, постоянная воздушная скорость



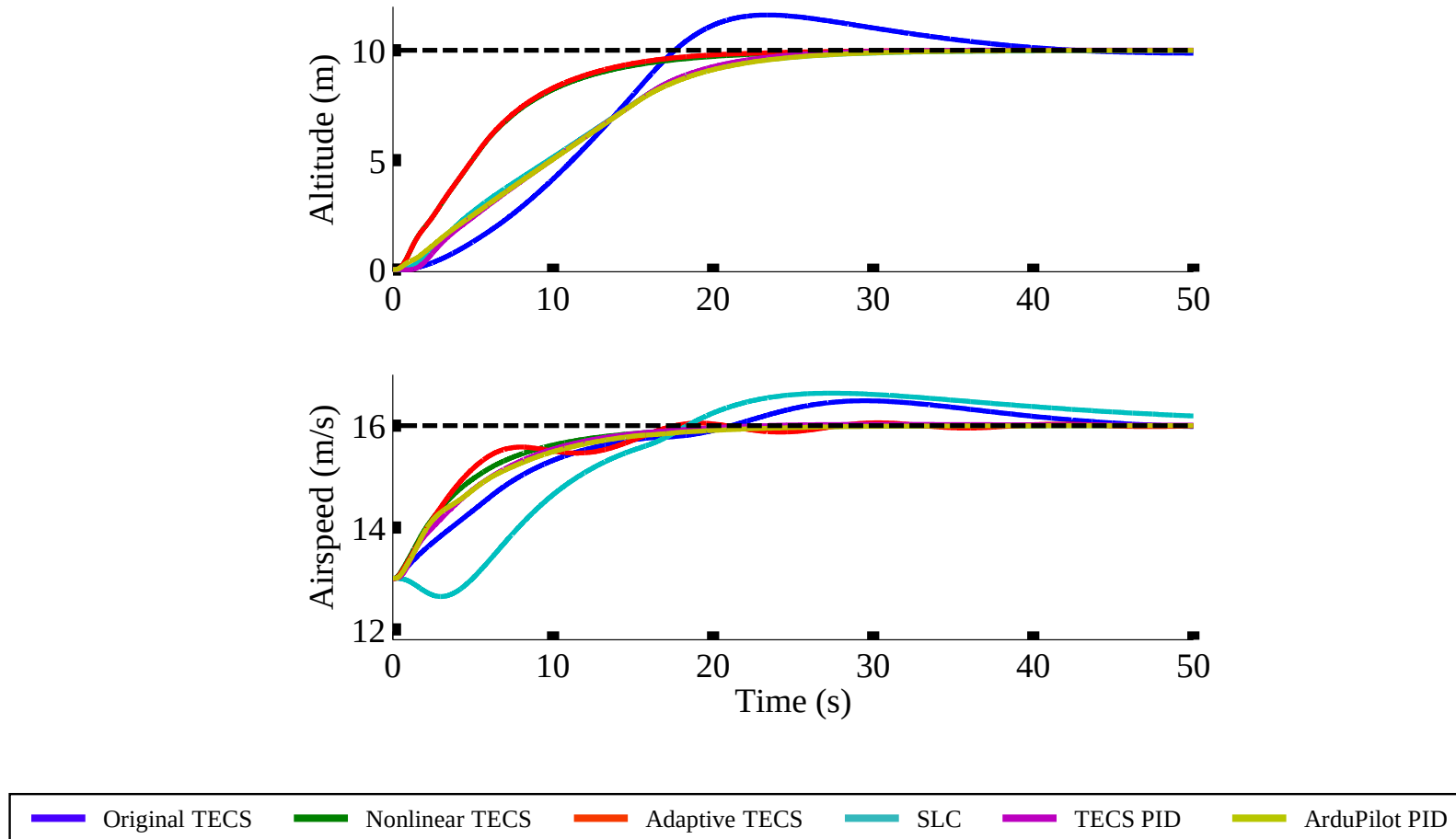
Управление полной энергией (TECS)

Ступенчатое воздействие по воздушной скорости, постоянная высота



Управление полной энергией (TECS)

Ступенчатое воздействие по высоте и воздушной скорости



Управление полной энергией (TECS)

- Наблюдения
 - TECS, по-видимому, работает лучше, чем последовательное замыкание контуров.
 - Устраняет необходимость в различных режимах полёта.
 - Нелинейная TECS представляется лучшей, но контроллер Ardupilot работает очень хорошо.

Итог

- Последовательное замыкание контуров
- Управление полной энергией
- **LQR-управление**

LQR-управление

Дополнение состояний интегратором.

Для системы в пространстве состояний

$$\dot{x} = Ax + Bu$$

$$z = Hx$$

где z — управляемый выход. Предположим, что цель — привести z к опорному сигналу z_r . Дополним состояние интегратором

$$x_I = \int_{-\infty}^t (z(\tau) - z_r) d\tau.$$

Следовательно,

$$\dot{x}_I = Hx - z_r = H(x - x_r)$$

LQR-управление

Дополнение состояний интегратором. (продолжение)

Определяя расширенное состояние как $\xi = (x^\top, x_I^\top)^\top$, получаем уравнения расширенной системы в пространстве состояний

$$\dot{\xi} = \bar{A}\xi + \bar{B}u,$$

где

$$\bar{A} = \begin{pmatrix} A & 0 \\ H & 0 \end{pmatrix} \quad \bar{B} = \begin{pmatrix} B \\ 0 \end{pmatrix}.$$

LQR-управление

Теория линейно-квадратичного регулятора
Для уравнения в пространстве состояний

$$\dot{x} = Ax + Bu$$

и симметричной положительно полуопределённой матрицы Q , и симметричной положительно определённой матрицы R , задача LQR состоит в минимизации функционала стоимости

$$J(x_0) = \min_{u(t)} \int_0^\infty x^\top(\tau)Qx(\tau) + u^\top(\tau)Ru(\tau)d\tau.$$

Если (A, B) управляема, а $(A, Q^{1/2})$ наблюдаема, то существует единственное оптимальное управление, которое задаётся в форме линейной обратной связи как

$$u^*(t) = -K_{lqr}x(t).$$

LQR-управление

Теория линейно-квадратичного регулятора (продолжение)

Коэффициент усиления LQR задаётся выражением

$$K_{lqr} = R^{-1} B^{\top} P,$$

где P — симметричное положительно определённое решение алгебраического уравнения Риккати

$$PA + A^{\top} P + Q - PBR^{-1}B^{\top}P = 0.$$

LQR-управление

Теория линейно-квадратичного регулятора (продолжение)

Следует отметить, что K_{lqr} — это оптимальные коэффициенты обратной связи при заданных Q и R . Регулятор настраивается путём изменения Q и R .

Обычно мы выбираем Q и R диагональными матрицами

$$Q = \begin{pmatrix} q_1 & 0 & \dots & 0 \\ 0 & q_2 & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & q_n \end{pmatrix} \quad R = \begin{pmatrix} r_1 & 0 & \dots & 0 \\ 0 & r_2 & \dots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & r_m \end{pmatrix},$$

где n — число состояний, m — число входов, $q_i \geq 0$ гарантирует, что Q положительно полуопределена, а $r_i > 0$ гарантирует, что R положительно определена.

LQR-управление

Автопилот бокового движения

Как выведено в Лекции 5, уравнения в пространстве состояний для бокового уравнения движения имеют вид

$$\dot{x}_{lat} = A_{lat}x_{lat} + B_{lat}u_{lat},$$

где $x_{lat} = (v, p, r, \phi, \psi)^\top$ и $u_{lat} = (\delta_a, \delta_r)^\top$.

Цель бокового автопилота — привести курс χ к командному курсу χ_c . Следовательно, мы дополняем состояние

$$x_I = \int (\chi - \chi_c) dt.$$

Поскольку $\chi \approx \psi$, мы аппроксимируем x_I как

$$x_I = \int (H_{lat}x_{lat} - \chi_c) dt,$$

где $H_{lat} = (0, 0, 0, 0, 1)$.

LQR-управление

Автопилот бокового движения (продолжение)

Следовательно, расширенные уравнения бокового состояния имеют вид

$$\dot{\xi}_{lat} = \bar{A}_{lat}\xi_{lat} + \bar{B}_{lat}u_{lat},$$

где

$$\xi_{lat} = (\tilde{v}, p, r, \phi, \tilde{\chi}, \int \tilde{\chi})^\top$$

$$\bar{A}_{lat} = \begin{pmatrix} A_{lat} & 0 \\ H_{lat} & 0 \end{pmatrix} \quad \bar{B}_{lat} = \begin{pmatrix} B_{lat} \\ 0 \end{pmatrix}$$

где

$$\tilde{v} = (V_a - V_a^c) * \sin \beta, \quad \tilde{\chi} = \chi - \chi^c$$

LQR-регулятор, спроектированный с использованием

$$Q = \text{diag}([q_v, q_p, q_r, q_\phi, q_\chi, q_I])$$

$$R = \text{diag}([r_{\delta_a}, r_{\delta_r}]).$$

Подсказки: Поскольку цель состоит в том, чтобы привести $\tilde{\chi}$ и $\int \tilde{\chi}$ к нулю, лучше задать малые веса для v , p и r по сравнению с другими переменными.

LQR-управление

Автопилот продольного движения

Как выведено в главе 5, уравнения в пространстве состояний для продольных уравнений движения имеют вид

$$\dot{x}_{lon} = A_{lon}x_{lon} + B_{lon}u_{lon},$$

где $x_{lon} = (u, w, q, \theta, h)^\top$ и $u_{lat} = (\delta_e, \delta_t)^\top$.

Цель продольного автопилота — привести высоту h к командной высоте h^c , а воздушную скорость V_a к командной воздушной скорости V_a^c . Следовательно, мы дополняем состояние

$$x_I = \begin{pmatrix} \int (h - h^c) dt \\ \int (V_a - V_a^c) dt \end{pmatrix} = \int \left(H_{lon}x_{lon} - \begin{pmatrix} h^c \\ V_a^c \end{pmatrix} \right) dt.$$

Следовательно,

$$\dot{x}_I = H_{lon}x_{lon} - \begin{pmatrix} h^c \\ V_a^c \end{pmatrix} = H_{lon}(x_{lon} - x^c)$$

где

$$H_{lon} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ \frac{1}{V_a} & \frac{1}{V_a} & 0 & 0 & 0 \end{pmatrix}.$$

LQR-управление

Автопилот продольного движения (продолжение)

Следовательно, расширенные уравнения продольного состояния имеют вид

$$\dot{\xi}_{lon} = \bar{A}_{lon}\xi_{lon} + \bar{B}_{lon}u_{lon},$$

где

$$\xi_{lon} = (\tilde{u}, \tilde{w}, q, \theta, \tilde{h}, \int \tilde{h}, \int \tilde{V}_a)^\top$$

$$\bar{A}_{lon} = \begin{pmatrix} A_{lon} & 0 \\ H_{lon} & 0 \end{pmatrix} \quad \bar{B}_{lon} = \begin{pmatrix} B_{lon} \\ 0 \end{pmatrix}$$

где

$$\tilde{u} = (V_a - V_a^c) \cos \alpha, \quad \tilde{w} = (V_a - V_a^c) \sin \alpha, \quad \tilde{h} = h - h^c, \quad \tilde{V}_a = V_a - V_a^c$$

LQR-регулятор, спроектированный с использованием

$$Q = \text{diag}([q_u, q_w, q_q, q_\theta, q_h, q_{Ih}, q_{IV_a}])$$

$$R = \text{diag}([r_{\delta_e}, r_{\delta_t}]).$$

Подсказки: Поскольку θ должно допускать ненулевые значения, и поскольку скорость тангажа не обязательно должна быть малой, веса q_θ и q_q следует задавать малыми.

Код на Python

```
import sys
from numpy import array, sin, cos, radians, concatenate, zeros, diag
from scipy.linalg import solve_continuous_are, inv
sys.path.append('.')
import parameters.control_parameters as AP
#from tools.transfer_function import TransferFunction
from tools.wrap import wrap
import chap5.model_coef as M
from message_types.msg_state import MsgState
from message_types.msg_delta import MsgDelta

class Autopilot:
    def __init__(self, ts_control):
        self.Ts = ts_control
        # initialize integrators and delay variables
        self.integratorCourse = 0
        self.integratorAltitude = 0
        self.integratorAirspeed = 0
        self.errorCourseD1 = 0
        self.errorAltitudeD1 = 0
        self.errorAirspeedD1 = 0
        # compute LQR gains
        CrLat = array([[0, 0, 0, 0, 1.0]])
        AAlat = concatenate((
            concatenate((M.A_lat, zeros((5,1))), axis=1),
            concatenate((CrLat, zeros((1,1))), axis=1)),
            axis=0)
        BBlat = concatenate((M.B_lat, zeros((1,2))), axis=0)
        Qlat = diag([.001, .01, .1, 100, 1, 100]) # v, p, r, phi, chi, intChi
        Rlat = diag([1, 1]) # a, r
        Plat = solve_continuous_are(AAlat, BBlat, Qlat, Rlat)
        self.Klat = inv(Rlat) @ BBlat.T @ Plat
        #CrLon = array([[0, 0, 0, 0, 1.0], [1.0, 0, 0, 0, 0]])
        CrLon = array([[0, 0, 0, 0, 1.0], [1/AP.Va0, 1/AP.Va0, 0, 0, 0]])
        AAlon = concatenate((
            concatenate((M.A_lon, zeros((5,2))), axis=1),
            concatenate((CrLon, zeros((2,2))), axis=1)),
            axis=0)
        BBlon = concatenate((M.B_lon, zeros((2,2))), axis=0)
        Qlon = diag([10, 10, .001, .01, 10, 100, 100]) # u, w, q, theta, h, intH
        Rlon = diag([1, 1]) # e, t
        Plon = solve_continuous_are(AAlon, BBlon, Qlon, Rlon)
        self.Klon = inv(Rlon) @ BBlon.T @ Plon
        self.commanded_state = MsgState()
```

```
def update(self, cmd, state):
    # lateral autopilot
    errorAirspeed = state.Va - cmd.airspeed_command
    chi_c = wrap(cmd.course_command, state.chi)
    errorCourse = saturate(state.chi - chi_c, -radians(15), radians(15))
    self.integratorCourse = self.integratorCourse + (self.Ts/2) * (errorCourse +
        self.errorCourseD1 = errorCourse
    xLat = array([[errorAirspeed * sin(state.beta)], # v
        [state.p],
        [state.r],
        [state.phi],
        [errorCourse],
        [self.integratorCourse]])
    tmp = -self.Klat @ xLat
    delta_a = saturate(tmp.item(0), -radians(30), radians(30))
    delta_r = saturate(tmp.item(1), -radians(30), radians(30))

    # longitudinal autopilot
    altitude_c = saturate(cmd.altitude_command,
        state.altitude - 0.2*AP.altitude_zone,
        state.altitude + 0.2*AP.altitude_zone)
    errorAltitude = state.altitude - altitude_c
    self.integratorAltitude = self.integratorAltitude \
        + (self.Ts/2) * (errorAltitude + self.errorAltitudeD
    self.errorAltitudeD1 = errorAltitude
    self.integratorAirspeed = self.integratorAirspeed \
        + (self.Ts/2) * (errorAirspeed + self.errorAirspeedD
    self.errorAirspeedD1 = errorAirspeed
    xLon = array([[errorAirspeed * cos(state.alpha)], # u
        [errorAirspeed * sin(state.alpha)], # w
        [state.q],
        [state.theta],
        [errorAltitude],
        [self.integratorAltitude],
        [self.integratorAirspeed]])
    tmp = -self.Klon @ xLon
    delta_e = saturate(tmp.item(0), -radians(30), radians(30))
    delta_t = saturate(tmp.item(1), 0.0, 1.0)

    # construct control outputs and commanded states
    delta = MsgDelta(elevator=delta_e,
        aileron=delta_a,
        rudder=delta_r,
        throttle=delta_t)
    self.commanded_state.altitude = cmd.altitude_command
    self.commanded_state.Va = cmd.airspeed_command
```